INDEX

NAME D.C. Bhargav Venkat SUBJECT POAI

STD. _____ DIV. _____ ROLL NO. 2410010032 SCHOOL REC

SR. NO.	~~PAGE NO.~~ mark	TITLE		DATE	TEACHER'S SIGN / REMARKS
1		Solve the 8-queens Problem	10	22/7/26	
2		Solve a graph using DFS	10	22/7/26	
3		Implement the MINIMAX ~~Diff~~ Algo.	10	29-7-26	
4		Implement A* Algorithm	10	29-7-26	
5		Implementation of Blocks world	10	5/8/26	
6		Implementing a Fuzzy Inference System		16/9/26	
7		Implementing Artificial Neural Network		23/9/26	
8		Implementing of Decision Tree		30/9/26	
9		Implemention of K-means Algorithm		7/10/26	
10		unification and Resolution		5/8/26	
11		Backward chaining		12/8/26	
12		Forward Detection		19/8/26	

completed
22/10/26

Date :- 22/7/26

Exp No :- 1

Solve the 8-Queens Problem.

Aim:

To place 8 queens on an 8 x 8 chessboard so that no two queens attack each other, No two queens share the same row, column, or diagonal.

Procedure:-

1. Create an 8 x 8 board initialized with 0s
2. Place a queen in each row by checking if its safe.
3. if no column is safe in a row, backtrack.
4. Repeat until all 8 queens are placed.

Python code:-

```
N=8
def print_solution(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print("\n").
def is_safe(board, row, col):
    for i in range(row):
        if board[i][col] or \
            col - (row-i) >=0 and board[i][col-(row-i)] or \
            col + (row-i) < N and board[i][col+(row-i)]:
            return False.
    return True
def solve_n-queens(board, row):
    if row == N:
        print_solution(board)
```

Date : 22/7/26

EXP NO :- 2

return True

found = False

for col in range(N):

if is_safe(board, row, col):

board[row][col] = 1

found = solve_n_queens(board, row+1) or found

board[row][col] = 0

return found

board = [[0] * N for _ in range(N)]

solve_n_queens(board, 0)

O/P :-

8-Queens Solution:

```
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .
```

Result :-

Thus the program of playing 8 queens on an 8X8 chessboard was executed successfully

22/7/26

Date = 22/7/26

EXP NO :- 2

Solve a Graph Using Depth First Search (DFS)

Aim:

To implement DFS traversal on a graph to explain all reachable nodes from a source.

Procedure:-

5) Define the graph using an adjacency list.

6) Start from the source node.

7) Recursively visit all unvisited neighbors.

Python code:

```
def dfs (graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add (start)
    Print (start, end=" ")
    for neighbor in graph [start]:
        if neighbor not in visited:
            dfs (graph, neighbor, visited)
graph = {
'A': ['B', 'C'],
'B': ['D', 'E'],
'C': ['F'],
'D': [],
'E': ['F'],
'F': []
}
```

Date:- 29/7/26

EX#NO:- 3

Implementi

Aim:-

To Use the M
the optimal u

Procedure:-

6. Define a
9. Apply recurs
10. Alternate bl

Python code:-

```
def Minimax(
    if depth ==h:
    return sco

If is Max:
    return max(
                Minimax
else:
    return min(mi
                minimax
scores = [3, 5, 6,
h = 3
Print("optimal va
```

```
Print ("DFS traversal from node A:")
dfs (graph, 'A')
```

O/P:-

```
DFS traversal from node A:
A B D E F C
```

Result:-

Thus the program of implementing DFS traversal on a graph to explore all reachable nodes from a source. Was executed successfully.

Date:- 29/7/26

EXP NO:- 3

Implement MINIMAX Algorithm

Aim:-

To Use the MINIMAX algorithm to evaluate the optimal move in a game tree.

Procedure:-

8. Define a list of terminal values.
9. Apply recursive MINIMAX from root to leaves.
10. Alternate b/w max & and min at each level.

Python code:-

```
def Minimax(depth, nodeIndex, isMax, scores, h):
    if depth == h:
        return scores[nodeIndex]
    if isMax:
        return max(minimax(depth+1, nodeIndex*2, False,
                           scores, h),
                   Minimax(depth+1, nodeIndex*2+1, False, scores, h))
    else:
        return min(minimax(depth+1, nodeIndex*2, True, scores, h),
                   minimax(depth+1, nodeIndex*2+1, True, scores, h))
scores = [3, 5, 6, 9, 1, 2, 0, -1]
h = 3
Print("optimal value:", minimax(0, 0, True, scores, h))
```

O/P:

```
Optimal value: 5
```

Result:

Thus the program of using the MINIMAX algorithm to evaluate the optimal move in a game tree was executed successfully.

29/4/26

Date:- 29/7/26

EXP NO:- 4

Implement A* Algorithm.

Aim:-

To find the shortest path from a source to a distination using the A* algorithm with heuristics.

Procedure:-

11. Initialize a priority queue with the start node.
12. maintain g(n) and h(n)
13. Expand the lowest f(n) = g(n) + h(n) node.
14. Track path using came_from dictionary.

Python code:-

```
from queue import priorityQueue

def a-star(graph, start, goal, heuristic):
    open_set = priorityQueue()
    open_set.put((0, start))
    came_from = {}
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    while not open_set.empty():
        current = open_set.get()
        if current == goal:
            path = []
            while current in came_from:
```

```
        path.append(start)
        return path[::-1]

    for neighbor, cost in graph[current]:
        tentative_g = g_score[current] + cost
        if tentative_g < g_score[neighbor]:
            came_from[neighbor] = current
            g_score[neighbor] = tentative_g
            f = tentative_g + heuristic[neighbor]
            open_set.put((f, neighbor))
    return None

path = a_star(graph, 'A', 'G', heuristic)
print("shortest path using A*:", path)
```

O/P :-

```
shortest path using A*: ['A', 'B', 'D', 'G']
```

Result :-

Thus, the program of using the MINIMAX algorithm to evaluate the optimal move in a game tree was executed successfully.

Date :- 29/7/26

EXP NO :- 5

Implementation of Blocks World

Aim:-

Simulate the Blocks world problem where a robot moves blocks from one configuration to another using planning.

Procedure:-

1. Define the initial and goal states.
2. Apply rules for block movement.
3. Use recursive planning.

Python code:-

```
initial_state = [['A', 'B'], ['C'], []]
goal_state = [['C', 'A'], ['B'], []]

def move (state, from_stack, to_stack):
    state [to_stack]. append (state[from_stack]. pop())
    return state.
def display_state (state):
    for i, stack in enumerate (state):
        Print (f" stack {i+1}: {stack}")
    Print()
Print ("Initial state:")
display_state (initial_state)

Print ("Final State:")
display_state (State)
```

Q9:-

```
Initial State:
Stack 1: ['A', 'B']
Stack 2: ['C']
Stack 3: []

Final State:
Stack 1: ['A', 'C']
Stack 2: ['B']
Stack 3: []
```

Aim:

To simulate the Blocks world problem where blocks from one stack are moved to another using planning.

Procedure:

1. Define the initial and goal states.
2. Apply the rules for block movement.
3. Use recursive planning.

Program code:

```
initial_state = [['A', 'B'], ['C'], []]
goal_state = [['A'], ['B'], ['C']]

def move(state, from_stack, to_stack):
    state[to_stack].append(state[from_stack].pop())
    return state

def plan(state, goal):
    if state == goal:
        return
    for i in range(len(state)):
        if state[i] and state[i] != goal[i]:
            print(f"Move {state[i][-1]} from Stack {i+1} to Stack {3}")
            move(state, i, 2)
            break
    plan(state, goal)

print("Initial State:")
print("Stack 1:", initial_state[0])
print("Stack 2:", initial_state[1])
print("Stack 3:", initial_state[2])

plan(initial_state, goal_state)

print("\nFinal State:")
print("Stack 1:", initial_state[0])
print("Stack 2:", initial_state[1])
print("Stack 3:", initial_state[2])
```

Result:

Thus, the code was executed successfully.

27/9/26

Date :- 16/9/26

ExpNo :- 6

Implementing a Fuzzy Inperance System

Aim:-

Create a fuzzy logic system to evaluate temperature and output fan speed.

Procedure:-

1. Define input/output variables.
2. Set membership functions.
3. Create rules and defuzzify output.

Python Code:-

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
temperature = ctrl.Antecendent(np.arange(0,41,1),
                                'Temperature')
fan_speed = ctrl.Consequent(np.arange(0,101,1), 'fan_speed')
temperature['cold'] = fuzz.trimf(temperature.universe, [0,0,20])
temperature['warm'] = fuzz.trimf(temperature.universe, [10,20,30])
temperature['hot'] = fuzz.trimf(temperature.universe, [20,40,40])
fan_speed['low'] = fuzz.trimf(fan_speed.universe, [0,0,50])
fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [25,50,75])
fan_speed['high'] = fuzz.trimf(fan_speed.universe, [50,100,100])
```

```
rule1 = ctrl.Rule(temperature['cold'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['warm'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['hot'], fan_speed['high'])
fan_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
fan_sim = ctrl.ControlSystemSimulation(fan_ctrl)
fan_sim.input['temperature']=30.
fan_sim.compute()
print("Fan speed:", fan_sim.output['fan_speed'])
```

O/P:

Result:-

Thus, the code was executed successfully.

16/9/26

Date :- 23/9/26

EXP NO :- 7

- Implementing Arti

Aim:-

To build a simple neu
digits.

Procedure :-

1. Load data.
2. Define a neural n
3. Train and evalu

Python code :-

```
from tensorflow.kera
from tensorflow.keras
from tensorflow.kera
from tensorflow.kera
(x_train, y_train), (x_test,
x_train, x_test = x_trai
x_train = to_categoric
y_test = to_categorica
model = sequential([
    Flatten(input_sh
    Dense(128, activa
    Dense(10, activation
])
```

Date :- 23/9/26

EXP NO :- 7

Implementing Artificial Neural Network

Aim :-

To build a simple neural network to classify handwritten digits.

Procedure :-

1. Load data.
2. Define a neural network.
3. Train and evaluate.

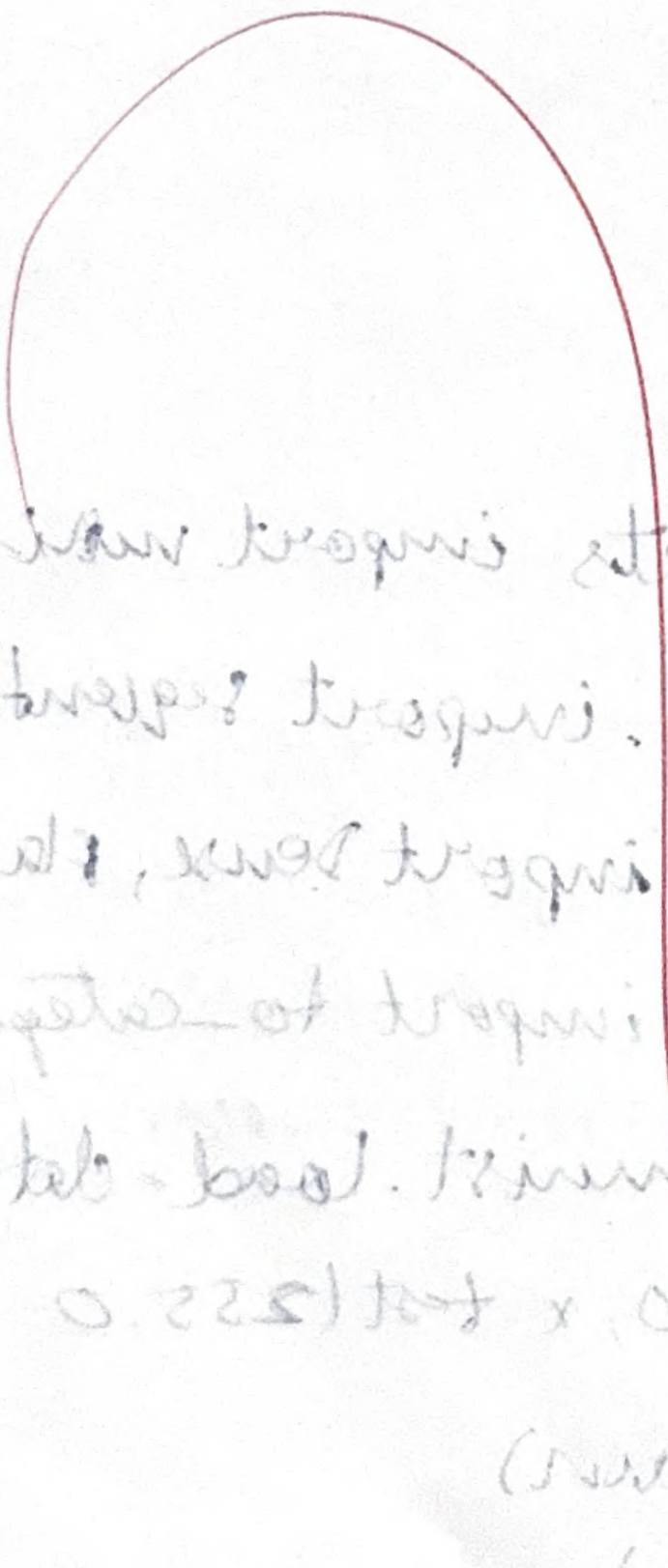
Python code :-

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train/255.0, x_test/255.0
x_train = to_categorical(y_train)
y_test = to_categorical(y_test)
model = sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, validation_split=0.1)
loss, acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", acc)
```

O/P :-

Accuracy: 0.8666666666667.



Result :-

Thus, the code was executed successfully.

23/9/26

Date :- 30/9/26

Exp No :- 8

Implementation of Decision Tree :-

Aim:-

To, Train a decision tree to classify iris flower species.

Procedure:-

1. Load dataset
2. Train the Decision Tree classifier.
3. Test and print accuracy.

Python code:-

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score.
data = load_iris()
x = data.data
y = data.target
x_train, x_test, y_train, y_test = train_test_split(
                                    x, y, test_size=0.2)
model = DecisionTreeClassifier()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
/usr/local/lib/python3.11/dist-
packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass
an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(**kwargs)
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8642 -
loss: 0.4699 - val_accuracy: 0.9648 - val_loss: 0.1215
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9611 -
loss: 0.1288 - val_accuracy: 0.9690 - val_loss: 0.1027
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9755 -
loss: 0.0857 - val_accuracy: 0.9747 - val_loss: 0.0867
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9822 -
loss: 0.0614 - val_accuracy: 0.9755 - val_loss: 0.0840
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9871 -
loss: 0.0443 - val_accuracy: 0.9765 - val_loss: 0.0797
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9730 -
loss: 0.0902
Test Accuracy: 0.9757000207901001
```

Accuracy: 0.8666666666666667

Result:

Thus, the Python code was executed successfully.

Date:- 7/10/26

EXP NO:- 9

Implementation of K-Means Algorithm

Aim:-

use k-means to cluster data points into groups.

Procedure:-

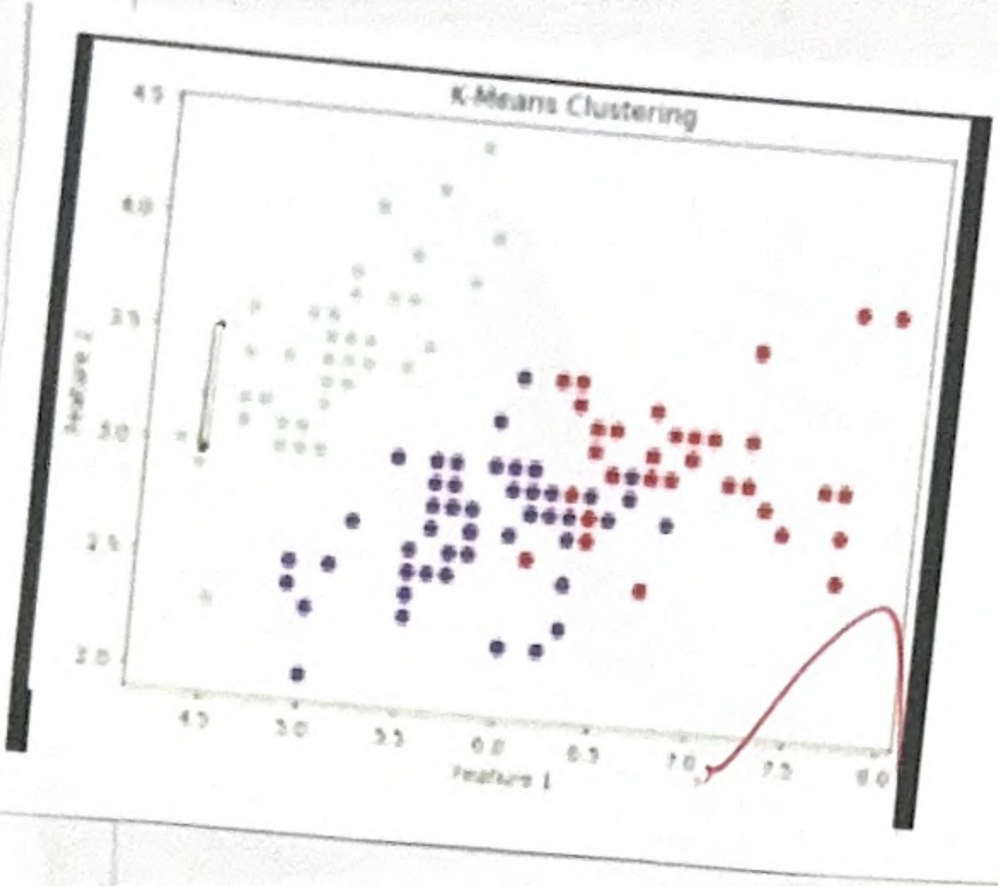
1. Load data.
2. Fit k-means
3. Plot or evaluate clusters.

Pythoncode:-

```
from sklearn.datasets import load_iris
from sklearn.clusters import kMeans
import matplotlib.pyplot as plt.

data = load_iris()
x = data.data
kmeans = kMeans(n_clusters = 3, random_state = 0)
kmeans.fit(x)
labels = kmeans.labels_
plt.scatter(X[:,0], x[:,1], c = labels, cmap = 'rainbow')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("k-means clustering")
plt.show()
```

O/P:-



Result:-

Thus, the code has executed successfully.

11/10/24

Date :- 5/8/26

Exp No :- 10

Unification & Resolution

Exercise 1 :- Sentence modelling

sentence (the cat sits).

sentence (the dog runs).

% query

% ? - sentence (X)

Exercise :2 :- Compound Term unification.

likes (john, food (Pizza, spices)).

likes (mary, food (Pizza, sweet)).

O/P :-

x = The_cat_sits : x = the dog_runs : false

O/P :- T = spices output :- true.

Result :-

Thus the program output has executed and verified successfully.

Date:- 12/8/26
EXP NO:- 11

Backward Chaining

Exercise 1:- Medical Diagonesis

% Facts
has_fever (John)
has_cough (John)

% Rule
has_flu (X) :- has_fever (X), has_cough

% query
% ?- has_fever (John)

Exercise 2:- student - Eligibility checker.

% Facts
Passed (alice, maths)
Passed (alice, science)

% Rule
Eligible (X) : Passed (X, maths), Passed (X, science)

% query
% ?- eligible (alice)

O/P:- True

Result :-
Thus, the program o/p has executed and verified successfully.

12/8/26

Date:- 19/8/26
EXP NO:- 12

Forward

Exercise 1:- Simple Infer

Dynamic knowns;
% Initial knowledge
start:- asserta (known (a)

% Rule
infer:- known (a)
assert (known (c)
< a, # 39:- >
% query
% 2 - start.

O/P

Inferred (

Result:-
Thus, the forward c
successfully.

12/8/26

Date :- 19/8/26

EXP NO :- 12

Forward Detection

Exercise1:- simple Inference

Dynamic known;

% - Initial knowledge

start :- asserta (known ear), assert (known(5)); infer

% Rule

infer :- known (a), known (6), % known (5)

assert (known (c)); write (a, ' is ', infered

<a, is>

% Query

% 2 start.

O/P:-

Inferred (

Executed

Result :-

Thus, the forward chaining is executed successfully.

19/8/26

completed

on 22/10/25